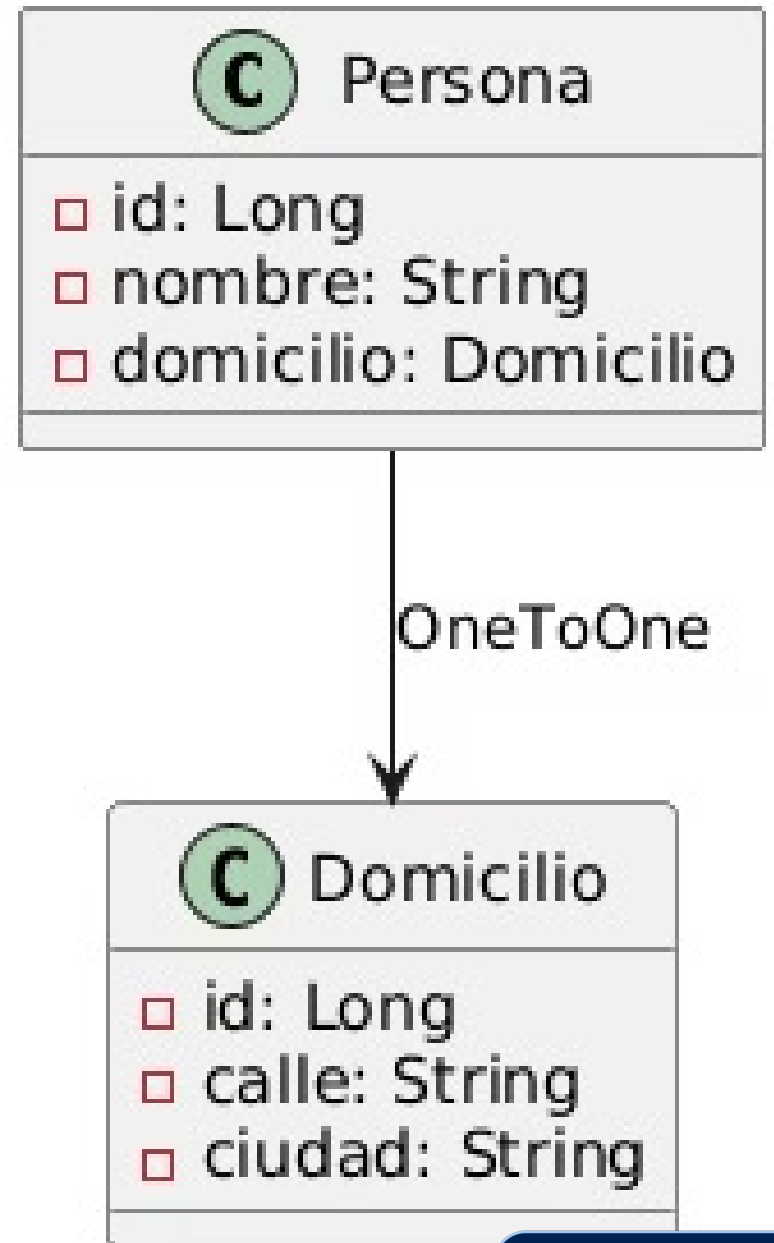


OneToOne – Relación Unidireccional

En este modelo, Persona conoce a Domicilio, pero la relación no es recíproca. La navegación va en un solo sentido.



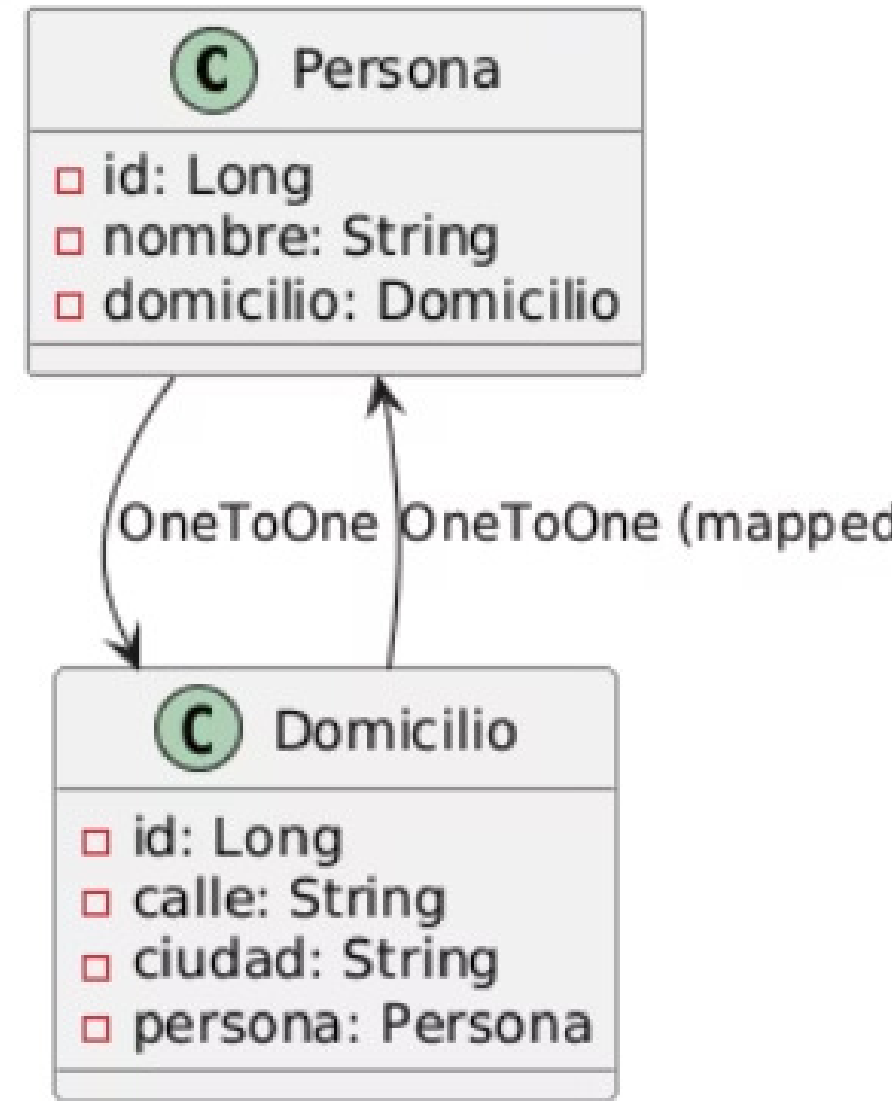
OneToOne – Relación Bidireccional

Ahora ambos lados conocen la relación. Persona tiene un Domicilio, y Domicilio sabe quién es su Persona.

Características

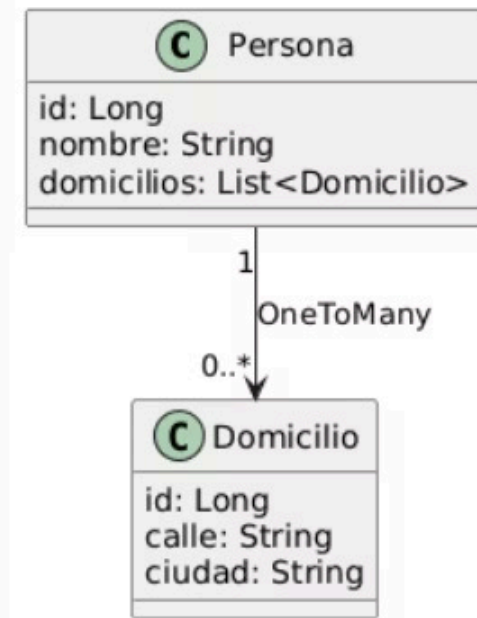
- Ambas entidades tienen referencias entre sí
- El atributo mappedBy indica quién es el lado inverso
- Persona sigue siendo dueña de la relación
- Domicilio es el lado no propietario (referenciado)

Nota importante: El lado con mappedBy (Domicilio) no es dueño de la relación. Esto significa que los cambios en esta referencia no afectan la base de datos directamente.



OneToMany – Relación Unidireccional

En este modelo, **Persona** conoce a varios **Domicilios**, pero los **Domicilios** no saben quién es su **Persona**. Es una relación en un solo sentido.

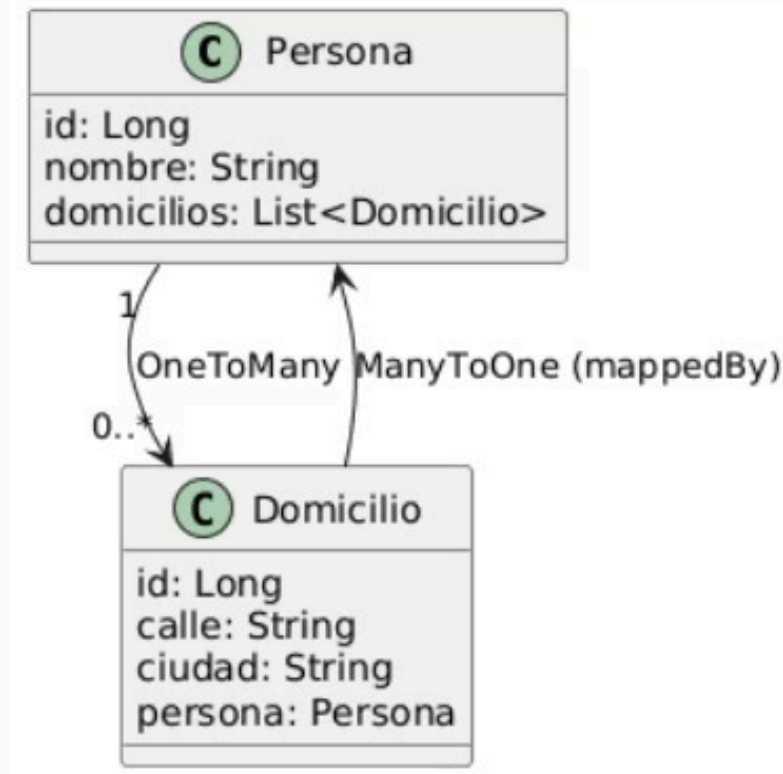


- ⊗ **Importante:** En este tipo de relación, la tabla **Domicilio** tiene la clave foránea (`id_persona`), aunque la relación esté mapeada desde **Persona**. Esto significa que la dirección de la relación en el modelo de objetos no siempre coincide con la ubicación de la clave foránea en la base de datos.



OneToMany – Relación Bidireccional

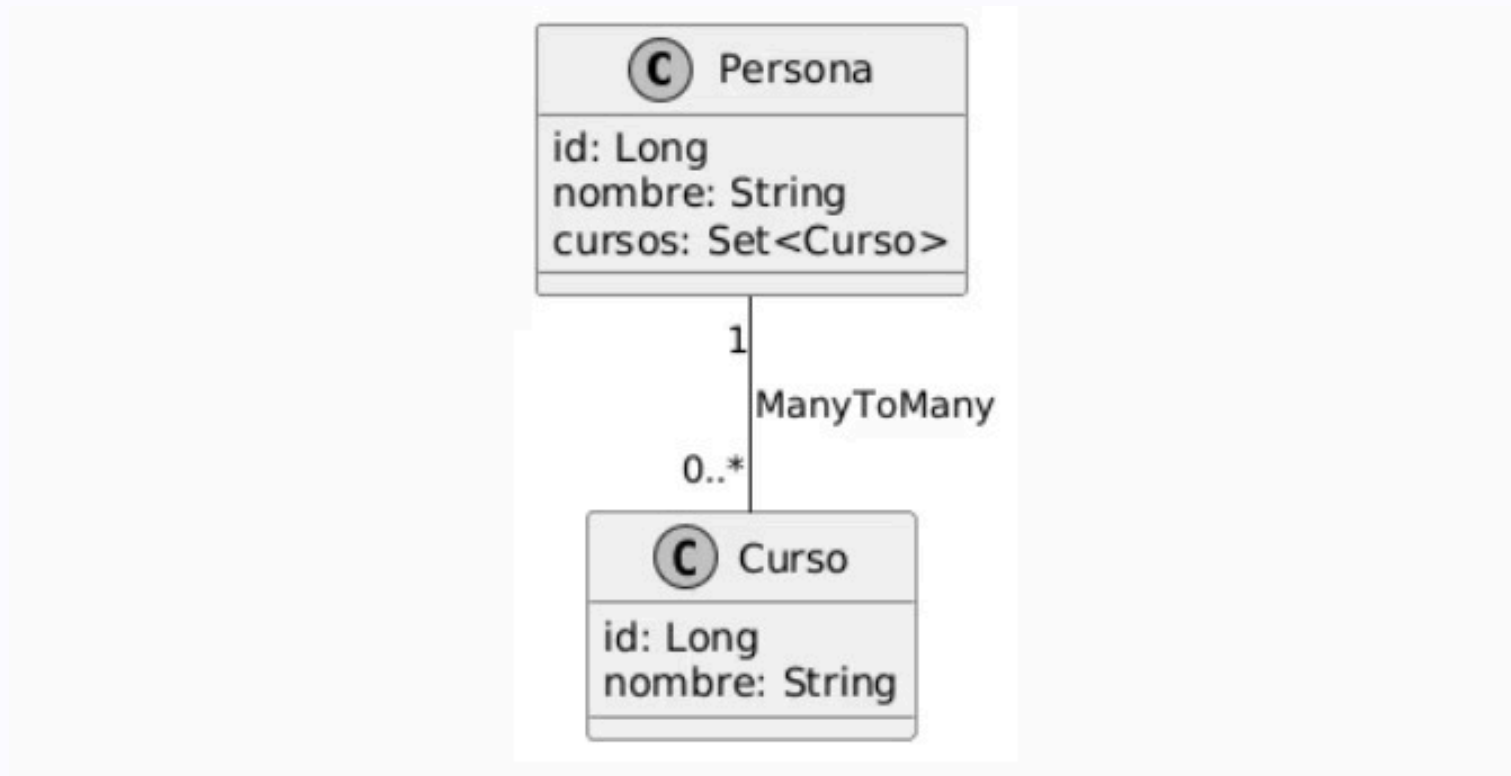
Ahora ambos lados conocen la relación: **Persona** tiene una lista de **Domicilios** y cada **Domicilio** sabe quién es su **Persona**.



- ⊗ **Importante:** Aunque **Domicilio** tiene la clave foránea (`person_id`), **Persona** es la dueña de la relación porque está del lado "One". El lado "Many" (**Domicilio**) usa el atributo `mappedBy` para indicar que es el lado inverso y que la relación es gestionada por **Persona**.

🎯 ManyToMany – Relación Unidireccional

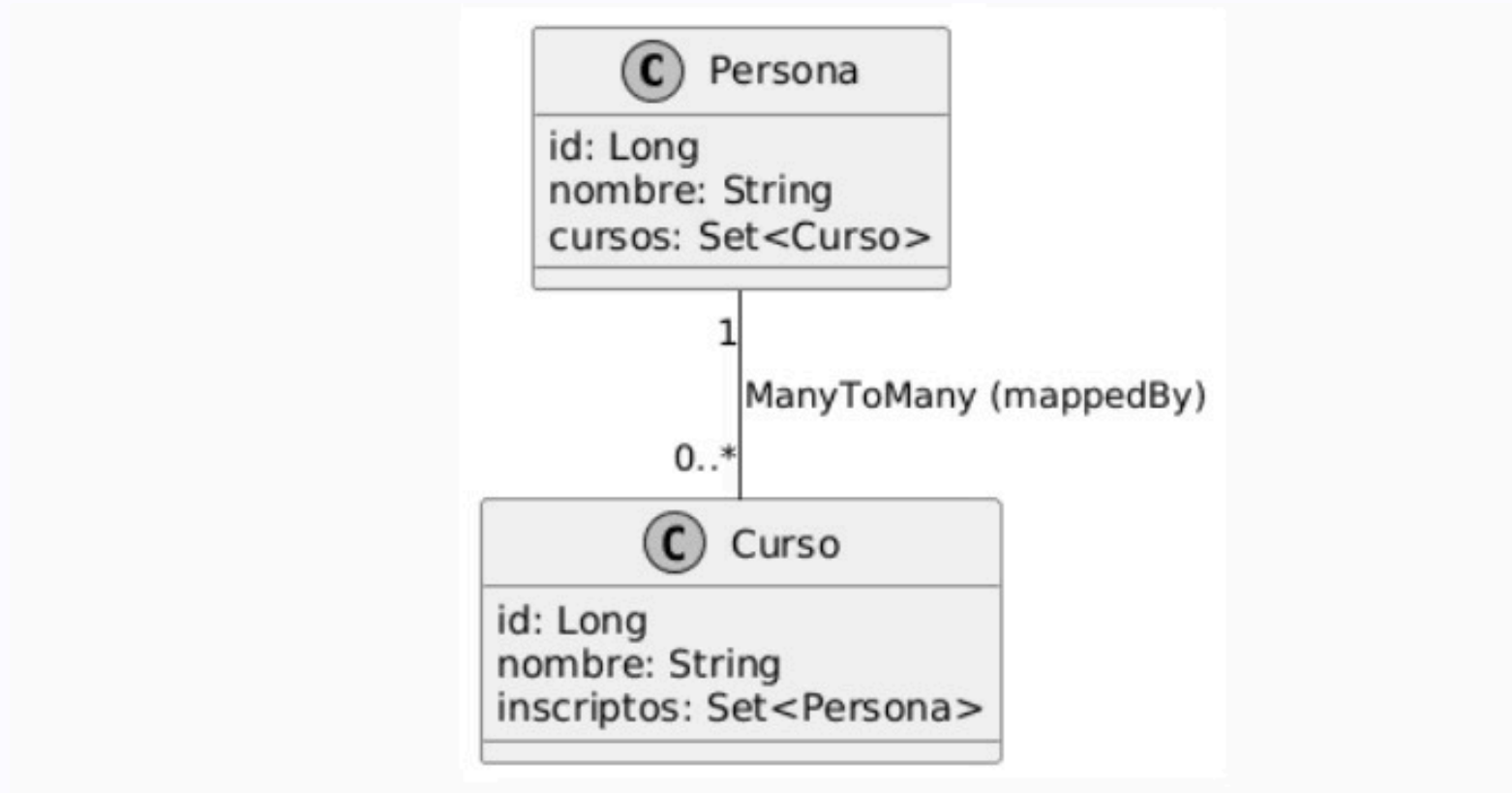
En este modelo, **Persona** conoce a varios **Cursos**, pero los cursos **no saben quién es su Persona**. Es una relación en un solo sentido donde la navegación va desde Persona hacia Curso.



- ⊗ **Importante:** En este tipo de relación, la tabla intermedia (`persona_curso`) contiene las claves foráneas hacia ambas entidades (`persona_id` y `curso_id`), pero **solo una entidad conoce la relación en el modelo de objetos**. En este caso, **Persona** es la entidad que gestiona la relación, aunque los datos de vinculación se almacenan en una tabla separada para mantener la integridad de la base de datos.

ManyToMany – Relación Bidireccional

Ahora ambos lados conocen la relación: **Persona** tiene un conjunto de **Cursos**, y cada **Curso** sabe quiénes son sus personas inscriptas.



- ❌ **Importante:** Aunque la tabla intermedia sigue siendo `persona_curso`, ahora también agregamos un atributo en **Curso**, con `mappedBy`, que indica que el dueño de la relación sigue siendo **Persona**. Esto permite una navegación completa en ambos sentidos, pero sin duplicar la tabla intermedia.